claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_c5395511-084\agent-ac760343fed0b0446.jsonl`
- SHA-256: `1cf963811311b2df734b28fed7a7b6a9e09b35196e4ebfc92b15050666b376c7`
- Source modified: `2026-07-06T21:29:24+00:00`
- Imported at: `2026-07-08T16:00:11+00:00`
- project: `wf_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:25:50.739Z)

```
Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-
preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there,
or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>  (db._apply_schema on
aiosqlite ":memory:").

WHAT F2 DOES:
- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video /
_content_item_to_video /
  get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of
content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences
(json.loads(policy.config_json)); TopicPreferenceStage
  (Tier 2): exclude-veto, include any/all, casefold substring match over source.title +
content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty);
_content_item_preview(...) builds a throwaway
  ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn,
account, source, item) loads the default
  policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now:
matched=engine.evaluate(); if matched and source:
  run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if
matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a
/set (F1's merge) and are read via
the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate:
ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium
(edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

DIMENSION: TopicPreferenceStage + config parsing correctness/edges (policy.py).
Investigate:
- Invalid/malformed topics config: TopicPreferenceStage returns StageResult.fail('invalid topic
config: ...') -> matched=False -> item DROPPED.
  So a bad topics config silently stops ALL delivery (fail-closed) with no surfaced reason. Is
```

```
that the right call vs fail-open (treat as disabled)?
  How could config become malformed given there is NO /set-topics command in F2 (topics only
settable via save_policy_config or manual edit)?
- parse_topic_preferences / _string_tuple edges: non-string include items, empty strings
(stripped), duplicate topics, mode casing, include as a
  non-list, topics as a non-dict. casefold substring matching false positives (e.g. include 'AI'
matches 'rain'/'again') ? acknowledged ADR v1 limit?
- load_topic_preferences does json.loads(policy.config_json) again (TopicPreferenceStage re-
parses) ? consistent with db._policy_config_json_to_dict?
  any divergence in what counts as a valid object?
- Empty haystack (no title, no caption text) with include topics -> fail (correct?). Exclude
with empty haystack -> no veto (correct?).
Run repros. Report findings.
```

2. assistant (2026-07-06T21:25:53.518Z)

```
I'll investigate the TopicPreferenceStage and config parsing correctness. Let me start by
reading the relevant files.
```

3. user (2026-07-06T21:25:54.070Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.
```

4. assistant (2026-07-06T21:25:55.915Z)

```
The path uses forward slashes. Let me find the actual worktree.
```

5. user (2026-07-06T21:25:58.737Z)

```
tg_video_filter
---
GIT FAIL
```

6. user (2026-07-06T21:26:02.534Z)

```
__init__.py
__main__.py
__pycache__
backfill.py
bootstrap.py
command_service.py
commands.py
config.py
db.py
delivery.py
filters
manager.py
models.py
policy.py
telethon_client.py
```

7. user (2026-07-06T21:26:05.450Z)

```
1       """Policy pipeline primitives for ADR 0012.
2
3       The policy layer is deliberately small and framework-free in F1: stages receive
4       an immutable context, return a structured result, and the evaluator stops on
5       the first failure. Later Track F phases add concrete stages.
6       """
7
8       from __future__ import annotations
9
10      import json
11      from dataclasses import dataclass, field
```

```
    from typing import Protocol

    from tg_video_filter.models import ContentItem, Policy, Source


    @dataclass(frozen=True)
    class PolicyContext:
        """Immutable snapshot available to a policy stage."""

        content_item: ContentItem
        source: Source
        policy: Policy
        fetcher: object | None = None
        budget: object | None = None


    @dataclass(frozen=True)
    class StageResult:
        """Outcome of one policy stage."""

        passed: bool
        reason: str
        stage_name: str
        metadata: dict[str, object] = field(default_factory=dict)

        @staticmethod
        def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=True,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )

        @staticmethod
        def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=False,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )


    class PolicyStage(Protocol):
        """A single stateless gate in the policy pipeline."""

        async def evaluate(self, ctx: PolicyContext) -> StageResult:
            """Evaluate ``ctx`` and return a stage result."""
            ...


    @dataclass(frozen=True)
    class TopicPreferences:
        """Parsed ADR 0012 topic preference config."""

        include: tuple[str, ...] = ()
        exclude: tuple[str, ...] = ()
        mode: str = "any"

        @property
        def enabled(self) -> bool:
            return bool(self.include or self.exclude)


```

```
    def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
        """Parse ``topics`` from a raw policy config object."""
        raw_topics = config.get("topics")
        if raw_topics is None:
            return TopicPreferences()
        if not isinstance(raw_topics, dict):
            raise ValueError("topics must be a JSON object")
        mode = raw_topics.get("mode", "any")
        if mode not in ("any", "all"):
            raise ValueError("topics.mode must be 'any' or 'all'")
        return TopicPreferences(
            include=_string_tuple(raw_topics.get("include")),
            exclude=_string_tuple(raw_topics.get("exclude")),
            mode=str(mode),
        )


    def load_topic_preferences(policy: Policy) -> TopicPreferences:
        """Load topic preferences from ``policy.config_json``."""
        decoded = json.loads(policy.config_json)
        if not isinstance(decoded, dict):
            raise ValueError("policy config_json must decode to a JSON object")
        return parse_topic_preferences(decoded)


    def _string_tuple(value: object) -> tuple[str, ...]:
        if value is None:
            return ()
        if not isinstance(value, list):
            raise ValueError("topic include/exclude values must be arrays")
        topics: list[str] = []
        for item in value:
            if not isinstance(item, str):
                raise ValueError("topic include/exclude arrays must contain strings")
            topic = item.strip()
            if topic:
                topics.append(topic)
        return tuple(topics)


    class TopicPreferenceStage:
        """ADR 0012 Tier 2 topic matching over source title and captured caption."""

        stage_name = "topic-preferences"

        async def evaluate(self, ctx: PolicyContext) -> StageResult:
            try:
                prefs = load_topic_preferences(ctx.policy)
            except (json.JSONDecodeError, ValueError) as exc:
                return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
            if not prefs.enabled:
                return StageResult.pass_(self.stage_name, "topic preferences disabled")

            haystack = _topic_haystack(ctx)
            matched_excludes = _matched_topics(prefs.exclude, haystack)
            if matched_excludes:
                return StageResult.fail(
                    self.stage_name,
                    f"excluded topic matched: {matched_excludes[0]}",
                    matched_exclude_topics=list(matched_excludes),
                )

            if not prefs.include:
                return StageResult.pass_(self.stage_name, "no include topics configured")

```

```
142             matched_includes = _matched_topics(prefs.include, haystack)
143             if prefs.mode == "any":
144                 if matched_includes:
145                     return StageResult.pass_(
146                         self.stage_name,
147                         f"matched include topic: {matched_includes[0]}",
148                         matched_include_topics=list(matched_includes),
149                     )
150                 return StageResult.fail(self.stage_name, "no include topics matched")
151 
152             missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
153             if missing:
154                 return StageResult.fail(
155                     self.stage_name,
156                     f"missing required topic: {missing[0]}",
157                     matched_include_topics=list(matched_includes),
158                     missing_include_topics=list(missing),
159                 )
160             return StageResult.pass_(
161                 self.stage_name,
162                 "matched all include topics",
163                 matched_include_topics=list(matched_includes),
164             )
165 
166 
167     def _topic_haystack(ctx: PolicyContext) -> str:
168         parts = [ctx.source.title or "", ctx.content_item.text or ""]
169         return "\n".join(parts).casefold()
170 
171 
172     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173         return tuple(topic for topic in topics if topic.casefold() in haystack)
174 
175 
176     async def evaluate_pipeline(
177         ctx: PolicyContext,
178         stages: list[PolicyStage],
179     ) -> tuple[bool, list[StageResult]]:
180         """Evaluate stages in order and short-circuit on the first failure."""
181         results: list[StageResult] = []
182         for stage in stages:
183             result = await stage.evaluate(ctx)
184             results.append(result)
185             if not result.passed:
186                 return False, results
187         return True, results
188 
189 
190     __all__ = [
191         "PolicyContext",
192         "PolicyStage",
193         "StageResult",
194         "TopicPreferenceStage",
195         "TopicPreferences",
196         "evaluate_pipeline",
197         "load_topic_preferences",
198         "parse_topic_preferences",
199     ]
200 
```

8. assistant (2026-07-06T21:26:08.741Z)

Now let me look at how `_policy_config_json_to_dict` works in db.py and compare validity semantics, plus check the wiring in telethon_client.py.

9. user (2026-07-06T21:26:11.627Z)

```
87:    config_json TEXT NOT NULL
125:    dedup_key            TEXT NOT NULL,
129:    UNIQUE (account_id, dedup_key)
136:    config_json TEXT NOT NULL,
259:# Pure function: computes the ``content_items.dedup_key`` value from metadata
263:# See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
266:def compute_dedup_key(
277:    """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
294:    ``dedup_key`` is worse than a loud failure at ingestion time.
307:                "Telegram dedup_key fallback requires source_id and provider_message_id "
314:            f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
480:                config_json TEXT NOT NULL
483:            "user_id, config_json",
566:             dedup_key, matched, seen_at)
589:        UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
591:          AND dedup_key LIKE 'tg:msg:%'
594:            WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
603:        UPDATE content_items SET dedup_key =
609:          AND dedup_key LIKE 'tg:msg:%'
615:              AND dedup_key LIKE 'tg:msg:%'
626:        INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
628:        SELECT user_id, 'default', config_json, 1,
699:                 CASE WHEN json_valid(config_json)
700:                      THEN json_extract(config_json, '$.delivery_target') END,
705:                   CASE WHEN json_valid(config_json)
706:                        THEN json_extract(config_json, '$.delivery_target') END,
730:                CASE WHEN json_valid(fcd.config_json)
731:                     THEN json_extract(fcd.config_json, '$.delivery_target') END,
878:        dedup_key=row["dedup_key"],
889:        config_json=row["config_json"],
1175:    The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1178:      - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1184:              size_bytes, mime_type, dedup_key, matched, seen_at)
1201:            item.dedup_key,
1227:async def get_content_item_by_dedup_key(
1228:    conn: aiosqlite.Connection, account_id: int, dedup_key: str
1230:    """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1233:    a single query: the ``dedup_key`` already encodes whichever tier was
1238:        "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1239:        (account_id, dedup_key),
1257:        """INSERT INTO policies (account_id, name, config_json, is_default,
1261:             config_json=excluded.config_json,
1267:            policy.config_json,
1295:def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1298:    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1304:        raise ValueError("policy config_json must decode to a JSON object")
1323:    return _policy_config_json_to_dict(policy.config_json)
1326:async def save_policy_config(
1337:            config_json=_policy_config_dict_to_json(config),
1516:    return _content_item_to_video(item, src, user_id, channel_id) if item else None
```

10. user (2026-07-06T21:26:15.715Z)

```
1295    def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1296        """Decode a policy config JSON object.
1297
1298        ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1299        expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1300        nested stage config can coexist.
1301        """
1302        decoded = json.loads(raw)
1303        if not isinstance(decoded, dict):
```

```
1304            raise ValueError("policy config_json must decode to a JSON object")
1305        return decoded
1306
1307
1308    def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1309        """Encode policy config deterministically and compactly."""
1310        return json.dumps(config, separators=(",", ":"), sort_keys=True)
1311
1312
1313    async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str,
object]:
1314        """Load the raw default policy config as a JSON object.
1315
1316        Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1317        as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1318        through the flat ``FilterConfig`` model.
1319        """
1320        policy = await get_default_policy(conn, account_id)
1321        if policy is None:
1322            return FilterConfig().model_dump(mode="json")
1323        return _policy_config_json_to_dict(policy.config_json)
1324
1325
1326    async def save_policy_config(
1327        conn: aiosqlite.Connection, account_id: int, config: dict[str, object]
1328    ) -> None:
1329        """Persist the raw default policy config object."""
1330        existing = await get_default_policy(conn, account_id)
1331        await upsert_policy(
1332            conn,
1333            Policy(
1334                id=existing.id if existing is not None else None,
1335                account_id=account_id,
1336                name=existing.name if existing is not None else "default",
1337                config_json=_policy_config_dict_to_json(config),
1338                is_default=True,
1339                created_at=(existing.created_at if existing is not None else _now()),
1340                updated_at=_now(),
1341            ),
1342        )
1343
1344
1345    # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1346
1347
1348    async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) ->
Destination:
1349        """Insert or update a destination row by (account, provider, target)."""
1350        await conn.execute(
1351            """INSERT INTO destinations
1352                 (account_id, provider, target, kind, title, is_output, created_at)
1353               VALUES (?, ?, ?, ?, ?, ?, ?)
1354               ON CONFLICT(account_id, provider, target) DO UPDATE SET
```

11. assistant (2026-07-06T21:26:18.165Z)

The `load_topic_preferences` and `_policy_config_json_to_dict` are consistent (both require dict). Now let me look at the telethon_client.py wiring.

12. user (2026-07-06T21:26:21.177Z)

```
45:    VideoRecord,
47:from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
172:      4. inserts a VideoRecord row,
270:        text = _message_text(message)
```

```
275:            topic_result = await _evaluate_topic_preferences(
279:                _content_item_preview(
289:            if topic_result is not None:
290:                reasons.append(topic_result.reason)
291:                matched = topic_result.passed
293:        record = VideoRecord(
374:def _message_text(message: object) -> str | None:
385:def _content_item_preview(
409:        dedup_key=db.compute_dedup_key(
423:async def _evaluate_topic_preferences(
433:    return await TopicPreferenceStage().evaluate(
470:    "_content_item_preview",
471:    "_evaluate_topic_preferences",
472:    "_message_text",
```

13. user (2026-07-06T21:26:24.437Z)

```
240                     log.info(
241                         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
242                         "channel=%s msg=%s original_channel=%s original_msg=%s",
243                         user.id,
244                         meta.size_bytes,
245                         meta.duration_s,
246                         meta.height,
247                         meta.width,
248                         channel_id,
249                         message_id,
250                         existing.channel_id,
251                         existing.message_id,
252                     )
253                     return
254
255             source = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
256
257             # Lazily record the channel (best-effort; failures are non-fatal).
258             try:
259                 channel = await db.upsert_channel(
260                     conn,
261                     Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
262                 )
263                 if channel.id is not None:
264                     source = await db.get_source_by_id(conn, channel.id)
265             except Exception:
266                 log.warning(
267                     "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
268                 )
269
270             text = _message_text(message)
271             result = engine_holder[ENGINE_KEY].evaluate(meta)
272             matched = result.matched
273             reasons = list(result.reasons)
274             if matched and source is not None:
275                 topic_result = await _evaluate_topic_preferences(
276                     conn,
277                     user.id,
278                     source,
279                     _content_item_preview(
280                         user.id,
281                         source,
282                         message_id,
283                         document_id,
284                         meta,
```

```
                        text,
                        matched,
                    ),
                )
                if topic_result is not None:
                    reasons.append(topic_result.reason)
                    matched = topic_result.passed

            record = VideoRecord(
                user_id=user.id,
                channel_id=channel_id,
                message_id=message_id,
                text=text,
                document_id=document_id,
                duration_s=meta.duration_s,
                width=meta.width,
                height=meta.height,
                size_bytes=meta.size_bytes,
                mime_type=meta.mime_type,
                matched=matched,
            )
            try:
                inserted = await db.insert_video(conn, record)
            except sqlite3.IntegrityError:
                # Lost a dedup race: a concurrent delivery of the same file
                # (same document_id or same message instance) won the insert.
                # The three-tier check above already classified this as new, so
                # the collision means another task is processing (or has
                # processed) it ? log and skip.
                log.info(
                    "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
                    user.id,
                    channel_id,
                    message_id,
                    document_id,
                )
                return
            log.info(
                "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
                user.id,
                channel_id,
                message_id,
                matched,
                reasons,
            )

            if matched:
                # Build link text for LINK/BOTH modes (None for FORWARD).
                link_text = None
                if delivery_mode != DeliveryMode.FORWARD:
                    channel_title = _channel_title(event)
                    link_text = build_link_text(
                        message_link=getattr(message, "link", None),
                        duration_s=meta.duration_s,
                        height=meta.height,
                        size_bytes=meta.size_bytes,
                        channel_title=channel_title,
                    )
                await deliver(
                    client,
                    message,
                    conn,
                    inserted.id,
                    delivery_mode,
                    link_text=link_text,
```

```
                    target=delivery_target,
                )

        return on_new_message


    # --- helpers -----------------------------------------------------------------


    def _extract_document_id(message: Message) -> int | None:
        """Extract Telegram's ``document.id`` from a video message.

        Returns None if the message has no document media or the id is absent.
        This is the most reliable content-based dedup key: Telegram reuses the
        same ``document.id`` when a file is forwarded across channels.
        """
        media = getattr(message, "media", None)
        if isinstance(media, MessageMediaDocument):
            media = media.document
        if isinstance(media, Document):
            return getattr(media, "id", None)
        return None


    def _message_text(message: object) -> str | None:
        """Return caption/message text normalized for storage, or None if empty."""
        text = getattr(message, "message", None)
        if text is None:
            text = getattr(message, "raw_text", None)
        if not isinstance(text, str):
            return None
        text = text.strip()
        return text or None


    def _content_item_preview(
        account_id: int,
        source: Source,
        message_id: int,
        document_id: int | None,
        meta: VideoMeta,
        text: str | None,
        matched: bool,
    ) -> ContentItem:
        """Build the ContentItem shape used by policy stages before insert."""
        assert source.id is not None
        return ContentItem(
            account_id=account_id,
            source_id=source.id,
            provider=source.provider,
```

14. user (2026-07-06T21:26:29.142Z)

```
            provider=source.provider,
            provider_message_id=str(message_id),
            content_type="video",
            text=text,
            document_id=str(document_id) if document_id is not None else None,
            duration_s=meta.duration_s,
            width=meta.width,
            height=meta.height,
            size_bytes=meta.size_bytes,
            mime_type=meta.mime_type,
            dedup_key=db.compute_dedup_key(
                provider=source.provider,
```

```
411                 document_id=str(document_id) if document_id is not None else None,
412                 source_id=source.id,
413                 provider_message_id=str(message_id),
414                 size_bytes=meta.size_bytes,
415                 duration_s=meta.duration_s,
416                 height=meta.height,
417                 width=meta.width,
418             ),
419             matched=matched,
420         )
421
422
423     async def _evaluate_topic_preferences(
424         conn: aiosqlite.Connection,
425         account_id: int,
426         source: Source,
427         content_item: ContentItem,
428     ) -> StageResult | None:
429         """Evaluate ADR 0012 topic preferences for a candidate item."""
430         policy = await db.get_default_policy(conn, account_id)
431         if policy is None:
432             return None
433         return await TopicPreferenceStage().evaluate(
434             PolicyContext(content_item=content_item, source=source, policy=policy)
435         )
436
437
438     def _peer_id(peer: object) -> int:
439         """Best-effort numeric id for the peer of a message.
440
441         Telegram represents channel/group/user peers as different TL types; we
442         collapse them to a single int for dedup/storage. The exact subclass isn't
443         important ? only that the id is stable per chat.
444         """
445         # Each Peer* type has a field like channel_id / chat_id / user_id.
446         for field in ("channel_id", "chat_id", "user_id"):
447             val = getattr(peer, field, None)
448             if val is not None:
```

15. assistant (2026-07-06T21:26:32.215Z)

Now I understand the wiring. Let me focus on the key concern from my dimension: the fail-closed behavior. Let me verify the critical scenario ? what happens when the default policy config is the initial one seeded at bootstrap. Let me check the seeded config and how `get_default_policy` config_json looks by default.

16. user (2026-07-06T21:26:33.568Z)

```
620
621         # 3. Backfill policies from filter_config: one 'default' policy per account
622         #    (Option 2A). is_default=1 so the legacy load_filter_config shim reads
623         #    this row. UNIQUE(account_id, name) dedups idempotent re-runs.
624         await conn.execute(
625             """
626             INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
627                                             created_at, updated_at)
628             SELECT user_id, 'default', config_json, 1,
629                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
630                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
631             FROM filter_config
632             """
633         )
634
635         # 4. Create the policies default-partial index (sig after backfill so the
636         #    UNIQUE constraint only applies once rows are populated). On fresh DBs
```

```
637         #    _SCHEMA creates it; on migrated DBs this is the first time.
638         await conn.execute(
639             "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
640             "ON policies(account_id) WHERE is_default = 1"
641         )
642 
643         # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
644         #    produce a content_items row. A mismatch signals fingerprint collisions
645         #    where two distinct videos shared (size,dur,h,w) ? the losers stay at
646         #    tg:msg: keys and are still present. Log a warning so the operator can
647         #    investigate; the data is preserved in *_deprecated tables.
648         async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
649             vc = (await cur.fetchone())[0]
650         async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
651             cic = (await cur.fetchone())[0]
652         if vc != cic:
653             import logging
654 
655             _logger = logging.getLogger(__name__)
656             _logger.warning(
657                 "migration_003: %d videos rows produced %d content_items rows "
658                 "(%d gap). Colliding items remain in videos_deprecated.",
659                 vc,
```

17. assistant (2026-07-06T21:26:37.090Z)

The default policy's config_json comes from filter_config, which is a valid JSON object (FilterConfig dumped). So no `topics` key -> `parse_topic_preferences` returns disabled -> pass. Good.

Now let me write repros to test the key edge cases in the parsing/stage logic.

18. assistant (2026-07-06T21:26:56.815Z)

```
import asyncio, json
from tg_video_filter.policy import (
    parse_topic_preferences, load_topic_preferences, TopicPreferenceStage,
    PolicyContext, TopicPreferences,
)
from tg_video_filter.models import ContentItem, Source, Policy

def mk_source(title):
    return Source(id=1, account_id=1, provider="telegram", provider_source_id="c1", title=title)

def mk_item(text):
    return ContentItem(
        account_id=1, source_id=1, provider="telegram", provider_message_id="1",
        content_type="video", text=text, dedup_key="tg:msg:1", matched=True,
    )

def mk_policy(cfg):
    return Policy(id=1, account_id=1, name="default", config_json=json.dumps(cfg),
                  is_default=True, created_at="t", updated_at="t")

async def stage_eval(cfg, title, text):
    stage = TopicPreferenceStage()
    ctx = PolicyContext(content_item=mk_item(text), source=mk_source(title),
policy=mk_policy(cfg))
    return await stage.evaluate(ctx)

async def main():
    # 1. casefold substring false positive: include 'AI' matches 'rain'
    r = await stage_eval({"topics": {"include": ["AI"]}}, "Weather: rain today", None)
    print("1 substring FP 'AI' in 'rain':", r.passed, "-", r.reason)
```

```
    r = await stage_eval({"topics": {"include": ["AI"]}}, "again and again", None)
    print("1b 'AI' in 'again':", r.passed, "-", r.reason)

    # 2. empty haystack + include -> fail
    r = await stage_eval({"topics": {"include": ["ai"]}}, None, None)
    print("2 empty haystack + include:", r.passed, "-", r.reason)

    # 3. exclude with empty haystack -> no veto -> pass
    r = await stage_eval({"topics": {"exclude": ["spam"]}}, None, None)
    print("3 exclude empty haystack:", r.passed, "-", r.reason)

    # 4. malformed: include not a list
    try:
        r = await stage_eval({"topics": {"include": "ai"}}, "ai news", None)
        print("4 include=string -> DROPPED:", r.passed, "-", r.reason)
    except Exception as e:
        print("4 raised:", type(e).__name__, e)

    # 5. mode casing
    try:
        p = parse_topic_preferences({"topics": {"mode": "ANY", "include": ["ai"]}})
        print("5 mode 'ANY':", p)
    except Exception as e:
        print("5 mode 'ANY' raised:", type(e).__name__, e)

    # 6. topics not a dict
    r = await stage_eval({"topics": ["ai"]}, "ai news", None)
    print("6 topics=list -> DROPPED:", r.passed, "-", r.reason)

    # 7. non-string include item
    r = await stage_eval({"topics": {"include": [123]}}, "ai news", None)
    print("7 include=[123] -> DROPPED:", r.passed, "-", r.reason)

    # 8. all-mode dedup / duplicate topics
    p = parse_topic_preferences({"topics": {"include": ["ai", "ai", "  ai  "], "mode": "all"}})
    print("8 dup include:", p.include)
    r = await stage_eval({"topics": {"include": ["ai", "ai"], "mode": "all"}}, "ai stuff", None)
    print("8b all-mode dup match:", r.passed, "-", r.reason)

    # 9. mode set but include empty, exclude present -> mode irrelevant, still runs
    r = await stage_eval({"topics": {"mode": "all", "exclude": ["spam"]}}, "spammy", None)
    print("9 mode+exclude only:", r.passed, "-", r.reason)

    # 10. include whitespace-only string stripped to empty -> disabled?
    p = parse_topic_preferences({"topics": {"include": ["   "]}})
    print("10 whitespace include enabled?:", p.enabled, p.include)

    # 11. mode non-string (e.g. number)
    try:
        p = parse_topic_preferences({"topics": {"mode": 1, "include": ["ai"]}})
        print("11 mode=1:", p)
    except Exception as e:
        print("11 mode=1 raised:", type(e).__name__, e)

    # 12. exclude wins even if include matches
    r = await stage_eval({"topics": {"include": ["ai"], "exclude": ["ai"]}}, "ai", None)
    print("12 include+exclude same:", r.passed, "-", r.reason)

    # 13. config_json not an object (JSON array at top level) via stage
    stage = TopicPreferenceStage()
    pol = Policy(id=1, account_id=1, name="default", config_json="[1,2]", is_default=True,
created_at="t", updated_at="t")
    ctx = PolicyContext(content_item=mk_item("ai"), source=mk_source("ai"), policy=pol)
    r = await stage.evaluate(ctx)
    print("13 config_json array -> DROPPED:", r.passed, "-", r.reason)
```

```
asyncio.run(main())
```

19. user (2026-07-06T21:26:56.816Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

20. assistant (2026-07-06T21:27:00.446Z)

That's a stale-path guard; the file doesn't exist yet. Let me use a different filename.

21. user (2026-07-06T21:27:18.673Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r.py", line 66, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r.py", line 26, in main
    r = await stage_eval({"topics": {"include": ["AI"]}}, "Weather: rain today", None)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r.py", line 22, in stage_eval
    ctx = PolicyContext(content_item=mk_item(text), source=mk_source(title),
policy=mk_policy(cfg))
~~~~~~~~~^^^^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r.py", line 18, in mk_policy
    return Policy(id=1, account_id=1, name="default", config_json=json.dumps(cfg),
                  is_default=True, created_at="t", updated_at="t")
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for Policy
created_at
  Input should be a valid datetime or date, input is too short [type=datetime_from_date_parsing,
input_value='t', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/datetime_from_date_parsing
updated_at
  Input should be a valid datetime or date, input is too short [type=datetime_from_date_parsing,
input_value='t', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/datetime_from_date_parsing
```

22. assistant (2026-07-06T21:27:22.087Z)

Let me check the Source/ContentItem/Policy model field names and fix.

23. user (2026-07-06T21:27:23.968Z)

```
123:class Source(BaseModel):
149:class ContentItem(BaseModel):
```

```
186:class Policy(BaseModel):
```

24. user (2026-07-06T21:27:26.385Z)

```
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169         text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
```

```
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
183         seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
```

25. assistant (2026-07-06T21:27:28.666Z)

Field name is `provider_source_id`. Let me fix the repro and use datetime.

26. user (2026-07-06T21:27:33.966Z)

```
1 'AI' in 'rain': True - matched include topic: AI
1b 'AI' in 'again': True - matched include topic: AI
2 empty haystack + include: False - no include topics matched
3 exclude empty haystack: True - no include topics configured
4 include=string -> DROPPED: False - invalid topic config: topic include/exclude values must be
arrays
5 mode 'ANY' raised: ValueError topics.mode must be 'any' or 'all'
6 topics=list -> DROPPED: False - invalid topic config: topics must be a JSON object
7 include=[123] -> DROPPED: False - invalid topic config: topic include/exclude arrays must
contain strings
8 dup include: ('ai', 'ai', 'ai')
8b all-mode dup: True - matched all include topics
10 whitespace include enabled?: False ()
11 mode=1 raised: ValueError topics.mode must be 'any' or 'all'
12 include+exclude same: False - excluded topic matched: ai
13 config_json array -> DROPPED: False - invalid topic config: policy config_json must decode to
a JSON object
14 all-mode missing one: False - missing required topic: ml
```

27. assistant (2026-07-06T21:27:41.384Z)

All my hypotheses confirmed. Now the key question: is the fail-closed a real regression? Let me verify what happens if a source is None or the whole flow ? specifically, confirm the caption text is the ONLY thing being matched and whether there is divergence in the "valid object" check. Both `load_topic_preferences` and `_policy_config_json_to_dict` require a dict ? consistent.

Let me confirm one important nuance for finding #8 (duplicate topics in all-mode): the `missing` computation uses `topic not in matched_includes`, and `matched_includes` is deduped-by-membership but preserves the include tuple with dups. Let me verify there's no false "missing" when duplicates exist. Already tested 8b passes. Good.

Now, the most important architectural finding is the **fail-closed drop**. Let me confirm the

ADR context and whether there's a surfaced reason. The reason is appended to `reasons` and logged, but the item is silently dropped (matched=False) with no user-facing error. Let me verify the ADR 0012 to see if fail-open was intended.

28. user (2026-07-06T21:27:43.660Z)

0012-policy-pipeline.md

C:/Users/avidu/Projects/_wt-pr51/docs/adr/0012-policy-pipeline.md

29. user (2026-07-06T21:27:48.018Z)

130:Topic matching is **captured-metadata-only** in v1: source title substring
219: serializes `FilterConfig` again, so unknown keys would be dropped. F1 must
263: F1 must avoid dropping pipeline keys when legacy config commands rewrite
265:- Topic matching is string-based substring matching in Phase 1 ? not
296:3. **Topic matching** ? substring only, or regex/glob? Recommend:
297:   substring (case-insensitive) for v1; natural-language matching in a

30. user (2026-07-06T21:27:51.334Z)

  }
}
```

`mode` controls include semantics: `"any"` means at least one include topic must match; `"all"` means every include topic must match. Excludes are always a veto. The pipeline itself remains AND-only across stages, but the Topic stage owns its internal include/exclude/OR logic and returns one `StageResult`.

Topic matching is **captured-metadata-only** in v1: source title substring match, captured message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue. Because the current live path does not populate `content_items.text`, F2 includes the ingestion work to capture `message.message`/caption text when inserting the `ContentItem`.

**Security stages ? Tier 3**

Security stages are **opt-in** and configured per-policy. The design defines two initial stages (implementation deferred to separate issues):

1. **URL/domain reputation**: check links in captured captions against a
   configurable blocklist. Metadata-only ? no page fetch.
2. **Caption text scanning**: reject captions containing configured unsafe
   terms or patterns.
  (metadata-only vs. I/O-heavy). Mitigated by keeping F1-F4 fetcher-free and
  making Tier 4's fetcher/budget explicit.
- Backwards compatibility is not automatic at the repository-shim level:
  F1 must avoid dropping pipeline keys when legacy config commands rewrite
  the default policy.
- Topic matching is string-based substring matching in Phase 1 ? not
  semantic. A future ML stage would replace it, but it's good enough for
  a v1.
- Caption and link stages require ingestion changes because the live path
  does not currently write `content_items.text` or `content_items.link_url`.
- Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget
  to avoid runaway costs.

## Alternatives considered

- **Do nothing; keep flat FilterConfig.** Rejected ? the umbrella exists
  to dissolve the conflation of quality, topic, and security into a
```

  single boolean. A pipeline is the natural generalization.
- **Inline each stage in the handler** (no protocol). Rejected ? makes
  it impossible to test stages independently, reorder them, or add new
  ones without touching the hot path.
- **Use a rules engine / DSL.** Rejected ? over-engineered for <100
  users. A Python list of callables is simpler, testable, and
  introspectable.
- **Bundle all phases into a single PR.** Rejected ? scope discipline.
  Each phase is independently shippable; this ADR defines the shared
  interface they code against.

Open questions for the decision owner

1. **`StageResult.metadata` shape** ? dict or typed per-stage? Recommend:
   dict for Phase 1 (flexible), typed in Phase 2 (once the shape
   stabilizes).
2. **Policy decisions table** ? store every decision, or only failures?
   Recommend: every decision (explainability), with a TTL-based cleanup
   for old traces.
3. **Topic matching** ? substring only, or regex/glob? Recommend:
   substring (case-insensitive) for v1; natural-language matching in a
   follow-up.
4. **Deep inspection budget** ? per-item byte limit or per-account
   bandwidth cap? Recommend: per-item limit (10 MB default) with a

31. assistant (2026-07-06T21:27:57.608Z)

ADR confirms substring case-insensitive matching is the intended v1 design (so the "AI" matches "rain" false-positive is an acknowledged limitation, not a bug ? low severity note at most). The ADR does not specify fail-open vs fail-closed for malformed config.

Now the critical assessment of fail-closed: In the current F2, the only ways config becomes malformed are `save_policy_config` (which takes a dict and json.dumps ? always valid dict-shaped) or manual DB edit. However, there's a subtler and more real path. Let me check whether `save_policy_config` or F1's `/set` merge could persist a `topics` value that parses fine as JSON but is structurally invalid (e.g. a user setting `topics` to a non-dict via some command). Let me check F1's config command path.

32. user (2026-07-06T21:28:00.326Z)

(Bash completed with no output)

33. assistant (2026-07-06T21:28:12.235Z)

No command sets topics in F2 (confirms the prompt: topics only settable via save_policy_config or manual edit). So malformed config is not reachable through any user command in this PR. The fail-closed is thus low-risk in practice for F2, but it's a latent design concern: when a `/set-topics` command lands later, a malformed topics config drops ALL delivery silently. That's a medium design-inconsistency note.

Let me now check one more real wiring concern: what if `matched` is already False from the engine ? the topic stage is skipped (good, short-circuit). But what about the `source is None` case ? topic stage skipped, so an item with topics configured but no source row gets delivered without topic filtering. Let me verify the exclude-veto bypass scenario. When `source is None`, topic filtering is entirely skipped (line 274). Let me check how often source is None on the live path.

Looking at lines 255-268: `source` is fetched, then re-fetched via upsert_channel. If both fail, source stays None and topic filtering is bypassed ? an exclude veto would NOT apply, delivering excluded content. But that's an edge (channel upsert failure) and the haystack would only be the caption text anyway (title unavailable). Let me note it as a lower-severity edge.

Now let me verify the most concrete finding for correctness: the `_topic_haystack` joins title and text with `\n`. Consider an exclude topic spanning the boundary ? not an issue since `\n`

separates. Fine.

Let me double check finding around `mode` being present but only exclude configured ? mode is validated even when include is empty (tested #9/#11 ? mode=1 raises even with no include). That means a config with `{"topics": {"mode": "invalid"}}` and NO include/exclude (effectively disabled) still fails-closed and drops everything. Let me verify.

34. user (2026-07-06T21:28:23.211Z)

A mode-invalid, empty include/exclude -> DROP ALL?: False - invalid topic config: topics.mode must be 'any' or 'all'
B typo key 'includes' -> treated as disabled -> pass: True - topic preferences disabled

35. assistant (2026-07-06T21:28:34.604Z)

Two additional confirmations:

- **A**: `mode` is validated eagerly even when no include/exclude is set. A config with only an invalid `mode` and no actual topics (semantically disabled) fails-closed and drops ALL delivery. The mode check happens before checking whether prefs are even enabled. This is over-eager validation.
- **B**: A typo in an include key (`"includes"` instead of `"include"`) is silently ignored ? the stage treats it as disabled and passes everything. Combined with fail-closed on the strict path, config errors have inconsistent outcomes: a typo'd key silently disables filtering (fail-open), while a typo'd `mode` value silently drops everything (fail-closed). This inconsistency is the real design smell.

I have enough to report. Let me compile findings, ranked by severity. The core defects in THIS PR's diff (policy.py):

1. **Fail-closed on malformed config drops ALL delivery silently** (medium ? latent, becomes high once `/set-topics` lands). No user-surfaced reason.
2. **Eager `mode` validation drops delivery even when topics are semantically disabled** (medium ? a config carrying only an invalid `mode` with no include/exclude ? which should be a no-op ? fails closed).
3. **Inconsistent fail-open vs fail-closed for config typos** (medium/low design-inconsistency): unknown keys silently disable filtering (fail-open) while invalid values fail-closed.
4. **Substring casefold false positives** ("AI" matches "rain"/"again") ? acknowledged ADR v1 limitation (low).
5. **`source is None` bypasses topic filtering entirely, including exclude vetoes** (low edge ? wiring in telethon_client, but it's the stage's exclude-veto being silently skipped).

Let me verify #5 is within this PR's diff (the `if matched and source is not None` guard is new in this PR).

36. user (2026-07-06T21:28:36.833Z)

done

37. assistant (2026-07-06T21:28:40.703Z)

The ADR doesn't address the source-None case. This is a genuine gap: exclude vetoes silently don't apply when the source row is absent. Given the code lazily creates the source (upsert_channel), source is almost always present, so this is a low-severity edge.

Now I'll submit my findings.

38. user (2026-07-06T21:29:24.582Z)

Structured output provided successfully